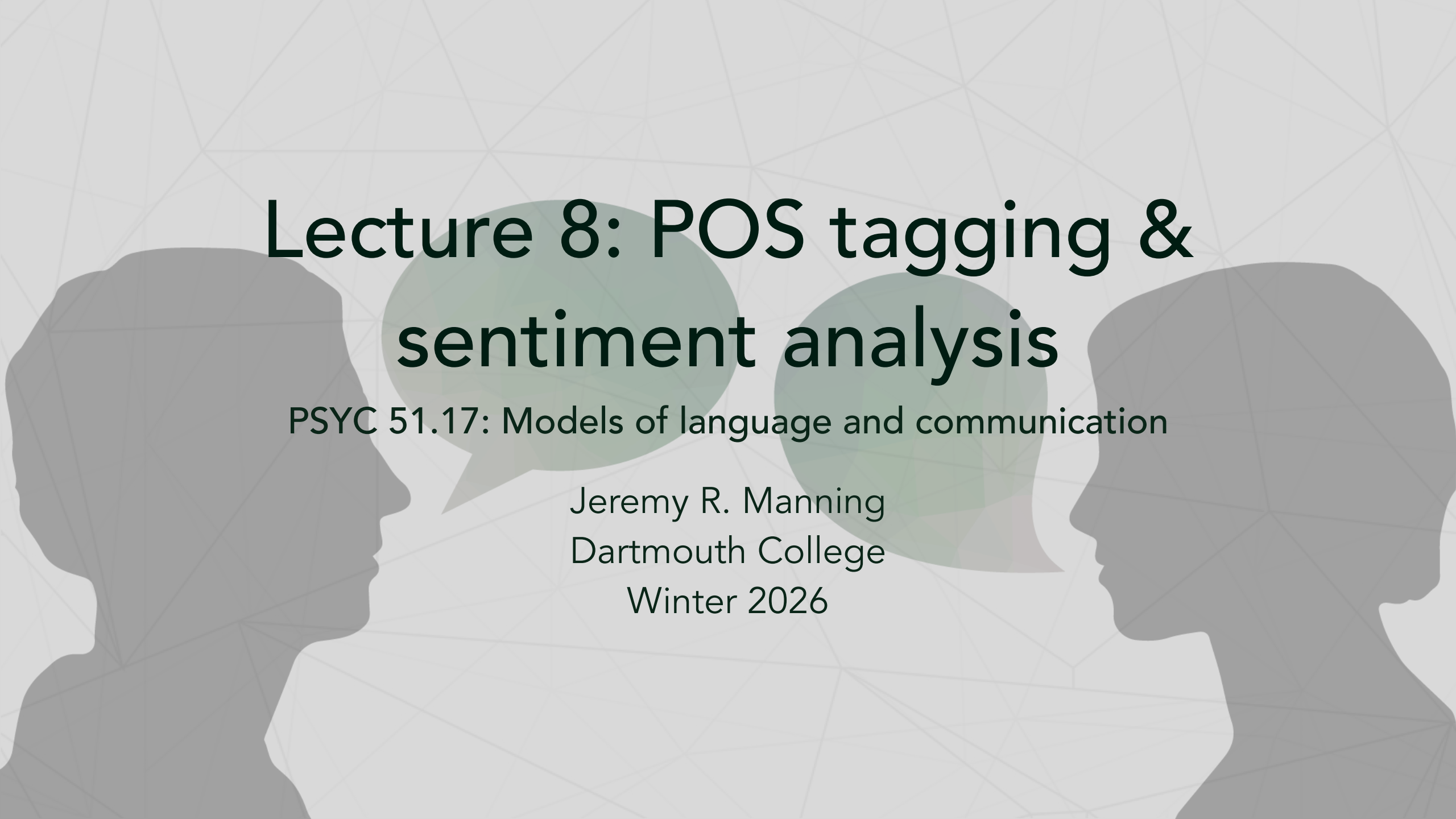




Lecture 8: POS tagging & sentiment analysis

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will

1. Understand part-of-speech (POS) tagging and its applications
2. Explore how neural networks learn grammatical structure
3. Apply sentiment analysis to real-world text
4. Fine-tune pre-trained models for domain-specific tasks
5. Critically evaluate whether models "understand" language

Central questions

- Can statistical patterns capture grammatical knowledge?
- What does it mean for a model to "understand" emotion?

Part-of-speech (POS) tagging

What is POS tagging?

- Assigning grammatical category to each word
- Categories: noun, verb, adjective, adverb, pronoun, preposition, etc.
- A fundamental NLP task

Example

1	The	cat	sat	on	the	mat
2	DET	NOUN	VERB	ADP	DET	NOUN

Why it matters

- Disambiguation: "book" as noun vs. verb
- Syntax parsing and understanding
- Information extraction, machine translation

POS tagsets: universal vs. fine-grained

Universal POS (17 tags):

- ADJ, ADV, ADP, AUX
- CONJ, DET, NOUN, NUM
- PRON, PROPN, VERB, ...

Penn Treebank (45+ tags):

- NN/NNS/NNP/NNPS (nouns)
- VB/VBD/VBG/VBN/VBP/VBZ (verbs)
- Much finer distinctions!

Trade-off

Simplicity vs. linguistic detail

Context matters: ambiguous words

Sentence	Word	POS	Explanation
"I read a book "	book	NOUN	Object being read
"Please book a table"	book	VERB	Action of reserving
"She runs fast "	fast	ADV	Modifies "runs"
"I will fast today"	fast	VERB	Action of not eating
"Please close the door"	close	VERB	Action
"Stay close to me"	close	ADV	Modifies position

Key insight

Context determines POS! Models must look at surrounding words.

spaCy resolves ambiguity using context

Code example

```
1 import spacy
2 nlp = spacy.load("en_core_web_sm")
3
4 sentences = [
5     "I need to book a flight",      # book = VERB
6     "I'm reading a great book",    # book = NOUN
7     "The record was broken",        # record = NOUN
8     "Please record the meeting",     # record = VERB
9 ]
10
11 for sent in sentences:
12     doc = nlp(sent)
13     for token in doc:
14         if token.text.lower() in ["book", "record"]:
15             print(f'"{sent}"')
16             print(f'  "{token.text}" → {token.pos_} ({spacy.explain(token.pos_)})\n')
```

Context helps to disambiguate parts of speech

Examples: "to book" vs "a book". Preceding words tell us whether "book" is a verb or noun.

What methods are used for POS tagging?

Traditional approaches (pre-neural network)

- Rule-based: hand-crafted grammar rules
- Hidden Markov Models (HMMs): probabilistic sequences
- Conditional Random Fields (CRFs): structured prediction

Modern neural network approaches

- Recurrent Neural Networks (RNNs/LSTMs): process sequences
- Transformers (BERT, etc.): bidirectional context
- Fine-tune pre-trained models on POS data

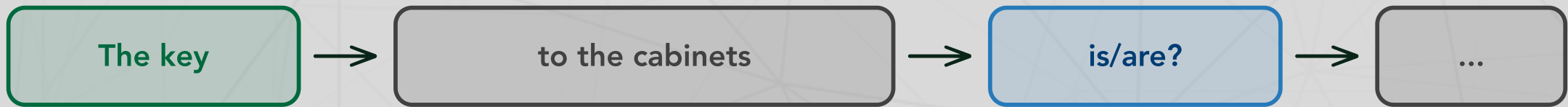
Advantages of neural network approaches

- Learn patterns from data (no hand-crafted rules)
- Capture long-range dependencies
- Handle ambiguity through context
- State-of-the-art accuracy (>97% on English!)

POS tagging is tricky!

Further reading...

Linzen, Dupoux, & Goldberg (2016, *TACL*). Assessing the ability of LSTMs to learn syntax-sensitive dependencies.



Challenge: distractor nouns between subject and verb

The challenge: which noun(s) go with the verb?

- Model must identify "key" (singular) as subject
- Ignore "cabinets" (plural distractor)
- Predict correct verb form "is" (not "are")

Token classification with HuggingFace

```
1  from transformers import pipeline
2  pos_tagger = pipeline("token-classification",
3                        model="vblagoje/bert-english-uncased-finetuned-pos",
4                        aggregation_strategy="simple")
5
6  sentence = "Apple Inc. is looking at buying a UK startup"
7  results = pos_tagger(sentence)
8
9  for result in results:
10     print(f"{result['word']:<15} {result['entity_group']:<8} ({result['score']:.3f})")
11  # Apple          PROPN      (0.998)
12  # Inc.           PROPN      (0.995)
13  # is             AUX        (0.999)
14  # looking        VERB       (0.997)
```

Further reading

[HuggingFace Chapter 7.2: Token Classification](#)

So what? Why is POS tagging important?

Applications

- Syntax parsing
- Named entity recognition
- Information extraction
- Machine translation
- Sentiment analysis (e.g., adjectives)

Linguistic insights

- Study grammatical structure
- Analyze language acquisition
- Explore cross-linguistic patterns
- Understand model "knowledge" of syntax
- Stylometric analysis

Think about it...

Consider how simple rule-based models like ELIZA parse user inputs versus what it means to explicitly model and identify parts of speech. Does POS tagging represent a deeper "understanding" of text? Or is it just another statistical pattern?

Try it out!

Check out the [POS tagging demo](#) to explore POS tagging interactively.

Sentiment analysis overview

What is sentiment analysis?

- Identifying emotional tone in text
- Classifying as positive, negative, neutral
- Extracting subjective information

Example

1	"I love this product!"	→ Positive
2	"This is the worst experience."	→ Negative
3	"It's okay, nothing special."	→ Neutral

Use of POS tagging

Sentiment is often linked to adjectives/adverbs (e.g., "great", "terrible"). POS tagging can help identify sentiment-bearing words. (Caveat: modern models often learn this implicitly!)

Sentiment analysis identifies emotional "tone"

Applications

- Social media monitoring
- Customer feedback analysis
- Market research
- Review analysis
- Political tracking
- Psychological studies (mood/emotion)
- Chatbots and virtual assistants

Granularity levels

- Binary: positive/negative
- Ternary: positive/negative/neutral
- Fine-grained: 1-5 stars
- Continuous: sentiment score
- Aspect-based: per-feature
- Emotion detection: joy, anger, sadness, etc.

Sentiment analysis challenges

Sarcasm and irony

- "Oh great, another meeting" (negative, despite "great")
- "This is the best movie I've ever fallen asleep to" (negative!)

Context-dependent sentiment

- "This movie is sick!" (positive in slang, negative literally)
- "The book was long" (neutral? negative?)

Mixed sentiment

- "Great food but terrible service" (both positive and negative)
- Aspect-based sentiment: food=positive, service=negative

Negation

- "not good" vs. "good"
- "I don't dislike it" (double negative = positive?)

Domain specificity

- "Explosive growth" (positive in business, negative in safety)
- Different domains have different sentiment patterns

Subtlety and ambiguity

- "It's okay, I guess" (neutral or slightly negative?)
- Subjective interpretations vary

Sentiment analysis with HuggingFace

Testing how models handle tricky cases

```
1 from transformers import pipeline
2 sentiment = pipeline("sentiment-analysis")
3
4 tricky_cases = [
5     ("Oh great, another Monday meeting", "NEGATIVE"),    # Sarcasm
6     ("This movie is not bad at all", "POSITIVE"),         # Negation
7     ("I don't dislike this product", "POSITIVE"),         # Double negative
8     ("Great camera but terrible battery life", "MIXED"),  # Mixed sentiment
9 ]
10
11 for text, expected in tricky_cases:
12     result = sentiment(text)[0]
13     print(f"{text}")
14     print(f"    Model: {result['label']} ({result['score']:.3f}) | Expected:
15         {expected}\n")
```

An unsolved problem...

Even modern models still struggle with sarcasm and complex negation!

Further reading

[Hugging Face Chapter 1.2: NLP Tasks with Pipeline](#)

Lexicon-based sentiment analysis

```
1  from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
2  analyzer = SentimentIntensityAnalyzer()
3
4  texts = ["I love this product! It's amazing!", "This is the worst
5           experience ever.",
6           "It's okay, nothing special.", "Great food but terrible
7           service!"]
8
9  for text in texts:
10     scores = analyzer.polarity_scores(text)
11     print(f"{text}")
12     print(f"  Pos: {scores['pos']:.2f}, Neg: {scores['neg']:.2f},
13           Compound: {scores['compound']:.3f}\n")
14 # VADER handles emoji and punctuation! "Great!!!" scores higher than
15 "Great"
```

Modern approach: Pre-trained models

Neural network advantages

- Learn context-dependent representations
- Capture word order and negation
- Handle sarcasm better (though still imperfect)
- Transfer learning: pre-train on large corpus, fine-tune for sentiment



Typical neural sentiment analysis architecture

Training

Fine-tune on labeled sentiment data (IMDb, Amazon reviews, etc.)

VADER vs Neural: Head-to-head comparison

Testing both approaches on the same examples

```
1  from vaderSentiment.vaderSentiment import  
   SentimentIntensityAnalyzer  
2  from transformers import pipeline  
3  
4  vader = SentimentIntensityAnalyzer()  
5  neural = pipeline("sentiment-analysis")  
6  
7  test_cases = [  
8      "I absolutely love this product!",  
9      "This is not what I expected, but in a good way",  
10     "The movie was so bad it was actually hilarious",  
11 ]  
12  
13 for text in test_cases:  
14     v_score = vader.polarity_scores(text)['compound']  
15     n_result = neural(text)[0]
```

continued...

VADER vs Neural: Head-to-head comparison

```
16 | print(f"Text: {text}")
17 | print(f"  VADER: {v_score:+.3f} ({'POS' if v_score > 0 else 'NEG'})")
18 | print(f"  Neural: {n_result['label']} ({n_result['score']:.3f})\n")
```

Key finding

Neural models handle nuance better, but VADER is faster and interpretable.

Sentiment analysis with HuggingFace

```
1 from transformers import pipeline
2 sentiment_analyzer = pipeline("sentiment-analysis")
3
4 texts = ["I love this product! It's amazing!", "This is the worst
5          experience ever.",
6          "It's okay, nothing special.", "I don't hate it, but I don't
7          love it either."]
8
9 for text in texts:
10     result = sentiment_analyzer(text)[0]
11     print(f"{text}")
12     print(f"  {result['label']}, Confidence: {result['score']:.3f}\n")
13 # "I love this product!" → POSITIVE (0.999)
```

Further reading

[HuggingFace Chapter 1.2: NLP Tasks](#)

Domain-specific sentiment models

Problem

General models miss domain-specific language

Solution

Fine-tune on domain-specific data!

Why it works: Domain-specific vocabulary ("bullish" in finance = positive), different sentiment expressions, adapted conventions.

Domain	Model	Data
Medical	BioBERT	Patient feedback
Twitter	TwitterBERT	Social posts
Products	RoBERTa	Amazon reviews
Movies	BERT	IMDb reviews

Comparing general vs. domain-specific

```
1  from transformers import pipeline
2  general = pipeline("sentiment-analysis")
3  financial = pipeline("sentiment-analysis", model="ProsusAI/finbert")
4
5  texts = ["The company's earnings exceeded expectations",
6           "Revenue declined but margins improved", "Stock prices
7           plummeted"]
8
9  for text in texts:
10     gen = general(text)[0]
11     fin = financial(text)[0]
12     print(f"{text}")
13     print(f"    General: {gen['label']} ({gen['score']:.3f}) | "
14           f"Financial: {fin['label']} ({fin['score']:.3f})\n")
15  # Financial model often more accurate for finance text!
```

Fine-tuning for sentiment analysis

Process overview

1. **Start with pre-trained model** (BERT, RoBERTa) — already knows language
2. **Prepare labeled dataset** — text + sentiment labels (pos/neg/neutral)
3. **Add classification head** — dense layer outputting class probabilities
4. **Fine-tune on sentiment data** — much faster than training from scratch!
5. **Evaluate** — accuracy, precision, recall, F1-score

Further reading

[HuggingFace Chapter 3.2: Processing Data for Fine-tuning](#)

Fine-tuning example (simplified)

```
1  from transformers import AutoModelForSequenceClassification, Trainer,  
   TrainingArguments  
2  
3  # 1. Load pre-trained model  
4  model = AutoModelForSequenceClassification.from_pretrained("bert-base-  
   uncased", num_labels=2)  
5  
6  # 2. Define training arguments  
7  training_args = TrainingArguments(output_dir="./results",  
   num_train_epochs=3,  
8      per_device_train_batch_size=16, evaluation_strategy="epoch")  
9  
10 # 3. Create Trainer and train (train_dataset, eval_dataset prepared  
   separately)  
11 trainer = Trainer(model=model, args=training_args)
```

Evaluation metrics for sentiment analysis

Beyond accuracy

- **Precision:** Of predicted positives, how many are truly positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall:** Of actual positives, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of precision and recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why not just accuracy?

- Imbalanced datasets (e.g., 90% positive reviews)
- Different costs for false positives vs. false negatives
- F1 gives balanced view of model performance

Confusion matrix example

	Predicted Positive	Predicted Negative
Actual Positive	TP = 90	FN = 10
Actual Negative	FP = 5	TN = 95

Metrics

- **Accuracy** = $(90 + 95) / 200 = 92\%$
- **Precision** = $90 / 95 = 95\%$
- **Recall** = $90 / 100 = 90\%$
- **F1** = $2 \times (0.95 \times 0.90) / (0.95 + 0.90) = 92\%$

Aspect-based sentiment analysis

Problem

Reviews often mention multiple aspects with different sentiments

Example

"The **food was delicious** but the **service was terrible**. The **atmosphere was okay**."

- Food: Positive
- Service: Negative
- Atmosphere: Neutral

Applications:

- Restaurant reviews: food, service, ambiance, price
- Product reviews: quality, price, shipping, customer service
- Hotel reviews: room, location, staff, cleanliness

Key insight

More nuanced than overall sentiment! Provides actionable insights.

Aspect-based sentiment: Worked example

```
1 review = ""The pasta was  
incredible - best I've had!  
2 However, we waited 45 min which  
was frustrating.  
3 Ambiance was nice but loud.  
Prices reasonable.""  
4  
5 aspects = {  
6     "food": ["pasta",  
7     "incredible"], # POSITIVE  
8     "service": ["waited",  
9     "frustrating"], # NEGATIVE  
10    "ambiance": ["nice", "loud"],  
    # MIXED  
    "price": ["reasonable"]  
    # POSITIVE  
}
```

Aspect	Sentiment
Food	POSITIVE
Service	NEGATIVE
Ambiance	MIXED
Price	POSITIVE

Actionable

Focus on improving wait times!

Real-world application: Product review analysis

Business value

- Identify product strengths and weaknesses
- Track sentiment trends over time
- Compare against competitors
- Prioritize product improvements



Product review analysis pipeline

Example insights

- **Product A:** Stable positive sentiment
- **Product B:** Declining sentiment → investigate quality issues!

Hands-on exercise

Try this yourself

1. Collect data:

- Scrape product reviews (Amazon, Yelp)
- Or use public dataset (IMDb, Twitter)

2. Compare approaches:

- Lexicon-based (VADER)
- General pre-trained model (HuggingFace pipeline)
- Domain-specific model (if available)

3. Analyze results:

- Where do models disagree?
- Which handles sarcasm better?
- Which is most accurate for your domain?

4. Bonus: Fine-tune a model on your specific dataset!

Discussion: Understanding emotion

Philosophical questions

1. Can models "feel" sentiment?

- They predict labels, but do they understand emotion?

2. Is sentiment objective or subjective?

- Different annotators may disagree on sentiment
- How do models handle ambiguity?

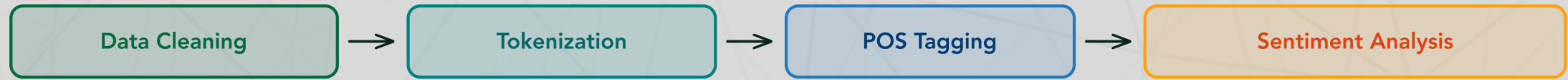
3. Cultural and linguistic variation:

- Sentiment expressions vary across cultures
- Can models capture these nuances?

4. Ethical considerations:

- Automated sentiment analysis in hiring, lending...
- Risks of bias and discrimination
- Should we trust model judgments?

Week 2: Each step builds on the previous one



The NLP pipeline

- **Data cleaning:** Remove noise
- **Tokenization:** Break into units
- **POS tagging:** Grammar structure
- **Sentiment:** Emotional meaning

Assignment 2: SPAM email classifier

Your task

Build a classifier to detect spam emails

Apply this week's concepts:

- **Data cleaning:** Remove HTML tags, normalize text
- **Tokenization:** Try different tokenizers (word, subword)
- **Features:** Extract useful signals (POS patterns, sentiment?)
- **Classification:** Train a model to identify spam

Think about:

- What makes spam different from legitimate emails?
- How does preprocessing affect accuracy?
- Can you use sentiment as a feature?
- What about POS patterns? (e.g., spam has more imperatives?)

Link

Assignment 2: SPAM classifier

Key takeaways

What we learned

1. **POS tagging reveals grammatical structure** — Neural nets learn syntax, but do they "understand" it?
2. **Sentiment analysis extracts emotional meaning** — Lexicons → neural networks; domain fine-tuning helps
3. **Statistical learning powers modern NLP** — Models learn patterns without explicit rules
4. **Context is crucial** — Words are ambiguous; Transformers excel at capturing context
5. **Critical thinking matters** — Question what "understanding" means; be aware of biases

Primary references

POS tagging and syntax:

- Linzen, Dupoux, & Goldberg (2016). Assessing the Ability of LSTMs to Learn Syntax-Sensitive Dependencies. *TACL*.
- Warstadt et al. (2020). BLiMP: The Benchmark of Linguistic Minimal Pairs. *TACL*.

Sentiment analysis:

- Pang & Lee (2008). Opinion Mining and Sentiment Analysis. *Foundations and Trends in IR*.
- Hutto & Gilbert (2014). VADER: A

HuggingFace resources:

- [Chapter 1.2: NLP Tasks with Pipeline](#)
- [Chapter 3.2: Processing Data for Fine-tuning](#)
- [Chapter 7.2: Token Classification](#)

Looking ahead: Weeks 3-4

Next topics

- Dimensionality reduction (PCA, UMAP)
- Bag-of-words and TF-IDF
- Word embeddings (Word2Vec, GloVe)
- Distributional semantics

Central question

"You shall know a word by the company it keeps"

How can we represent word *meaning* computationally?

Prepare by

- Completing Assignment 2
- Thinking about: What is "meaning"? How would you define it?
- Exploring: Vector representations and semantic similarity

Additional resources

Tools and libraries:

- spaCy: <https://spacy.io/>
- HuggingFace Transformers: <https://huggingface.co/docs/transformers>
- VADER Sentiment: <https://github.com/cjhutto/vaderSentiment>

Datasets:

- IMDb Movie Reviews: <https://ai.stanford.edu/~amaas>
- Amazon Product Reviews: <https://registry.opendata.aws/amazon-reviews/>
- Stanford Sentiment Treebank: <https://nlp.stanford.edu/sentiment>

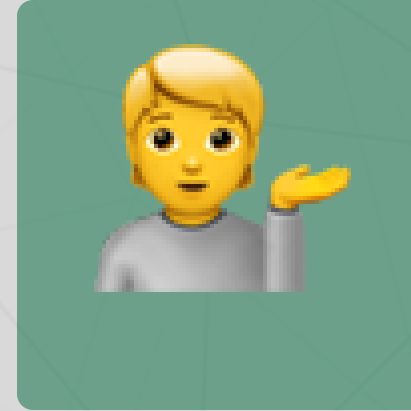
Questions? Want to chat more?



Email me



Join our Discord



Come to office hours

Congratulations!

Week 2 complete! See you in Week 3!